

R1.01 - Initiation au développement

Fiche TD 6

En 2015, les supermarchés Auchan offrent à leurs clients la possibilité de collectionner 135 cartes relatives aux héros Disney-Pixar. Ces cartes sont destinées à être rassemblées dans un collecteur (Figure 1). Un collecteur est composé de 135 petits compartiments plastiques. Chaque compartiment plastifié est destiné à recevoir un ou plusieurs exemplaire(s) d'une même carte offerte.

Ainsi, par exemple, le compartiment n°127 du collecteur est destiné à recevoir la carte TILT ou plusieurs cartes TILT si le client en a reçu plusieurs (Figure 3). Le compartiment n°1 contiendra le ou les exemplaires de la carte JOIE (Figure 2) en possession du collectionneur.

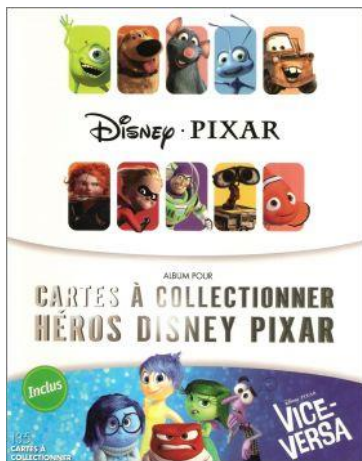


Figure 1



Figure 2

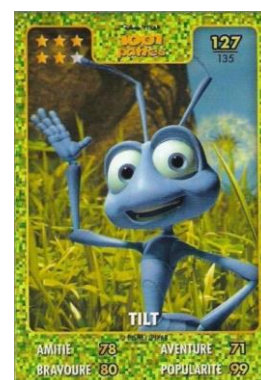


Figure 3

- Relever les informations qu'il est pertinent de mémoriser au sujet d'une carte.
- Proposer la structure de données **une_carte** qui représente ces données.
- Proposer la structure de données **les_cartes** qui représente l'organisation du collecteur (on doit, grâce à cette structure, connaître les détails de la carte destinée à chaque compartiment du collecteur).

Un client peut ne posséder aucun exemplaire d'une carte. Mais il peut également avoir collecté un exemplaire, voire plusieurs exemplaires d'une carte.

- Proposer la structure de données **une_collection** qui modélise la collection d'un client.
- Écrire la procédure **cartes_manquantes** qui **affiche la liste des cartes manquantes** dans la collection d'un client. Toutes les caractéristiques relevées sur les cartes sont affichées : chaque ligne d'affichage correspond à une carte.
- Écrire la fonction **exemplaires_multiples** qui **compte le nombre de compartiments où figurent des cartes en plusieurs exemplaires** dans la collection d'un client.

- g. Écrire la fonction booléenne *collection_complète* qui **vérifie si la collection d'un client est complète**. Une collection est complète si chacune des 135 cartes y figure **au moins** en un exemplaire.
- h. Écrire la fonction *recherche_carte* qui **recherche la position d'une carte précise** qui figure dans le collecteur.

Exemple : On recherche la carte Tilt dans le collecteur.

Pierre donne la totalité des cartes de sa collection à Quentin. Ce dernier ajoute les cartes ainsi reçues dans les compartiments concernés de sa collection.

- i. Écrire la procédure *réunir_collections* qui **fusionne les deux collections**.

Dans le magasin, les cartes sont distribuées dans des pochettes de 3 cartes (Figure 4).

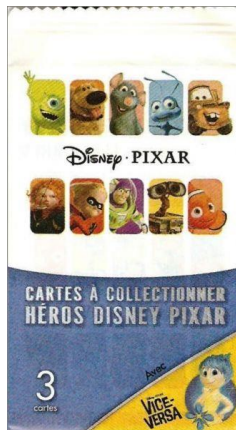


Figure 4

- j. Proposer la structure de données **une_pochette** qui représente une pochette de 3 cartes. Cette structure de données doit être **la plus simple et économique possible** et ainsi **éviter la redondance d'informations**.

Un client reçoit une pochette de 3 cartes.

- k. Écrire la procédure *ajout_pochette* qui **ajoute les 3 cartes d'une pochette à la collection du client**.

On dispose d'une procédure *generation_pochette* qui génère aléatoirement le contenu d'une pochette : *procedure generation_pochette*(var nouvelle_pochette : une_pochette).

Un client reçoit n pochettes de cartes (n entier positif strictement supérieur à 0) et les ajoute à sa collection.

- l. Écrire la procédure *ajout_plusieurs_pochettes* qui **ajoute le contenu de l'ensemble de ces n pochettes de cartes à la collection du client**.

Un client perd une carte.

- m. Écrire la procédure *retrait_carte* qui **gère la perte d'une carte dans une collection**.
Exemple : Le client a perdu un exemplaire de la carte Tilt.

Les clients s'organisent afin de réaliser des échanges. Par exemple, deux clients veulent échanger les cartes Tilt et Joie. Une carte s'échange toujours contre une seule carte. Un client ne cède une carte que s'il possède plus **d'un** exemplaire de cette carte.

- n. Écrire la fonction booléenne *comparaison_collections* qui **teste si deux collections sont identiques**. Deux collections sont identiques si une même carte figure dans les deux collections (peu importe le nombre d'exemplaires) ou si cette carte ne figure dans aucune des deux collections.
- o. Écrire la fonction *vérification_faisabilite_echange* qui **vérifie si deux clients peuvent échanger deux cartes**.
- p. Écrire une procédure *echange_cartes* qui réalise **l'échange de deux cartes de deux collections lorsque celui-ci est possible**.